

FPGA Experiment Report

Experiment 5 — PS/2 Keyboard Interface

Task: FPGA 5 — Keyboard (PS/2 interface)

Student 1: Mahmood Stitia

ID: 214032682

Student 2: Saleh Khalil

ID: 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
Ps2_Interface.v	Done	PS/2 receiver; testbench passes
Ps2_Interface_tb.v	Done	Self-checking; all scenarios pass
Ps2_Display.v	Done	7-seg + LED strobe (CDC to 100 MHz)
Ps2_Top.v	Done	Top level, FPGA pin wiring
task1.xdc	Done	Pin + clock constraints

Issues

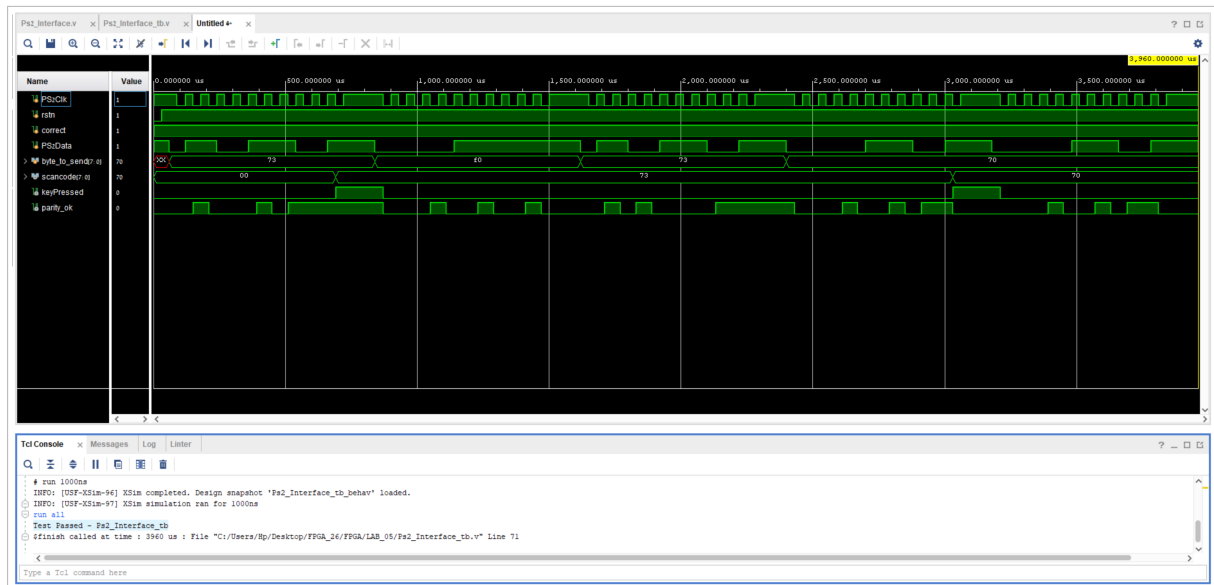
Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- Ps2_Interface.v:
- Ps2_Interface_tb.v:
- Ps2_Display.v:
- Ps2_Top.v:
- task1.xdc:

Part 2 — Simulations and answers

This part contains the simulation of `Ps2_Interface` with annotated waveforms. (*Per the PDF, only `Ps2_Interface.v` is simulated.*)

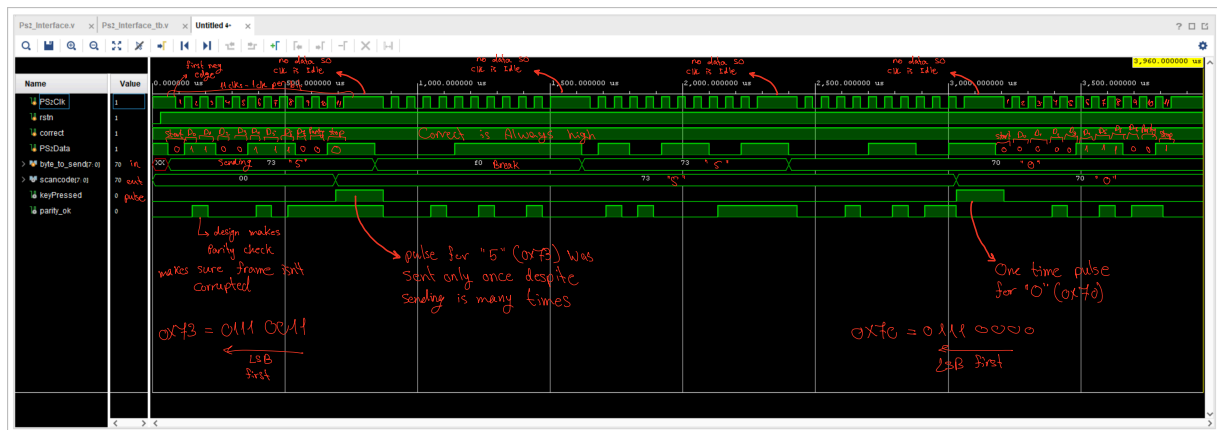
Simulation — Ps2_Interface



Full `Ps2_Interface_tb` simulation (signals `PS2Clk`, `rstn`, `correct`, `PS2Data`, `byte_to_send[7:0]`, `scancode[7:0]`, `keyPressed`, `parity_ok`). The `correct` flag stays high for the whole run and the Tcl console reports `Test Passed - Ps2_Interface_tb ($finish at 3940 μs)`.

Signals (top to bottom in the waveform):

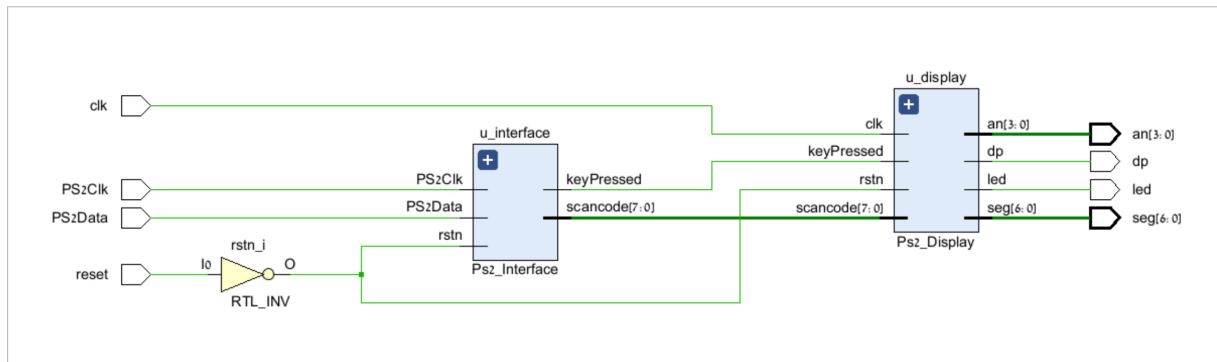
- `PS2Clk` — the keyboard clock (input). It toggles only while a byte is being sent and is idle-high otherwise; the interface shifts data in on its *falling* edge.
- `rstn` — active-low *asynchronous* reset (driven by the test-bench). When low it clears the interface even while `PS2Clk` is idle.
- `correct` — a test-bench-only pass/fail flag. It is AND-ed with the expected result after every step and stays high only if all checks passed (then the console prints `Test Passed`).
- `PS2Data` — the serial data line (input). Carries the 11-bit frame (start, 8 data LSB-first, parity, stop), sampled on the falling edge of `PS2Clk`.
- `byte_to_send[7:0]` — a test-bench register holding the byte currently being bit-banged onto `PS2Data`; it shows which key code is on the wire at each moment.
- `scancode[7:0]` — interface output: the most recent accepted make-code, latched until the next valid press.
- `keyPressed` — interface output: a one-cycle pulse on the first make-code of a new press (not on auto-repeats and not on `0xE0/0xF0` prefixes).
- `parity_ok` — internal interface signal (exposed in the bench via `uut.parity_ok`): high when the frame's odd-parity bit matches the 8 data bits; `keyPressed` is asserted only when `parity_ok` is high.



Annotated close-up of the same simulation. The red notes mark the first key edge and that the first bit is sent on the negative clock edge; that *correct* stays high the whole time; the byte values $0x73 = 0111\ 0011$ and $0x70 = 0111\ 0000$ (sent LSB-first); the break ($0xF0$) sequence; and that *keyPressed* gives a single pulse per press even though the make-code is re-sent many times.

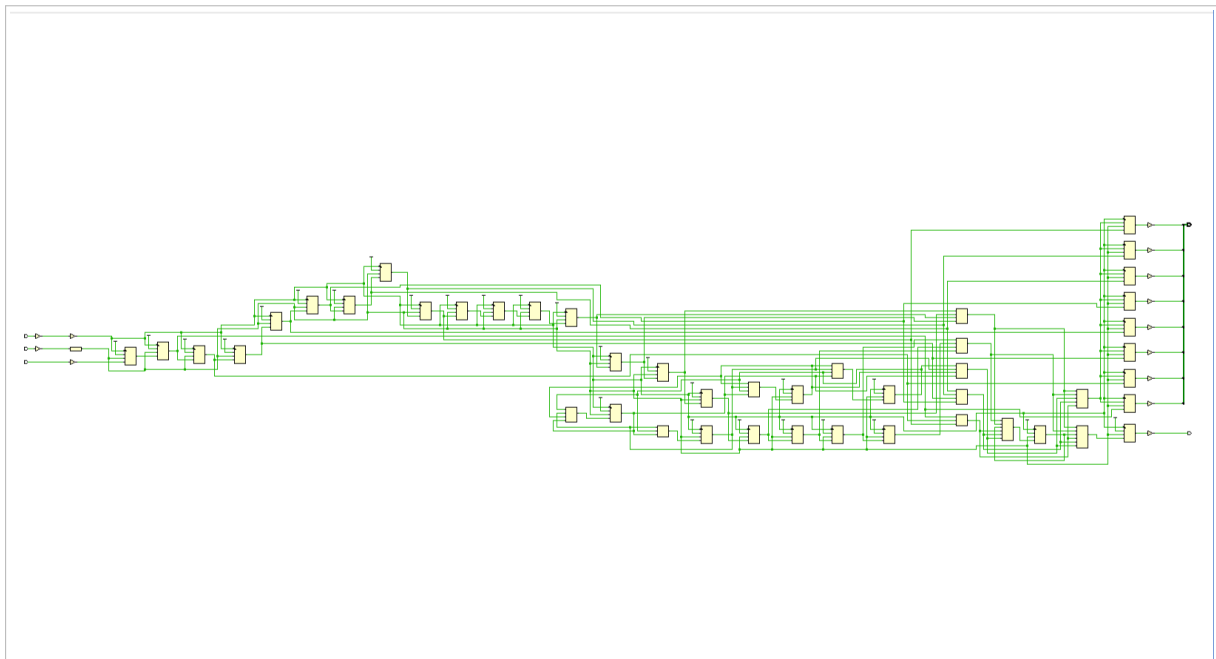
Elaborated Design

Top module



RTL-elaborated schematic of *Ps2_Top*: *u_interface* (*Ps2_Interface*, *PS2Clk* domain) feeds *keyPressed* and *scancode[7:0]* into *u_display* (*Ps2_Display*, 100 MHz domain); the reset push-button passes through an *RTL_INV* (*rstn_i*) into *rstn*. Inputs *clk*, *PS2Clk*, *PS2Data*, *reset* are on the left and outputs *an[3:0]*, *dp*, *led*, *seg[6:0]* on the right.

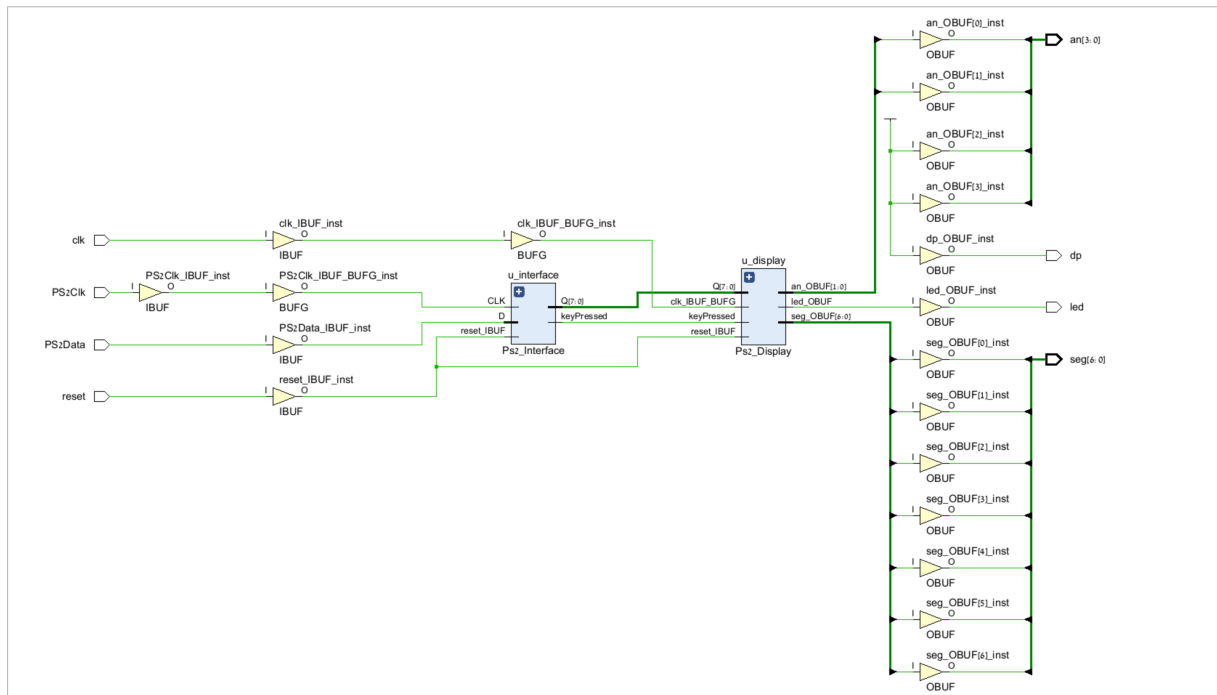
Ps2_Interface



RTL-elaborated schematic of the Interface

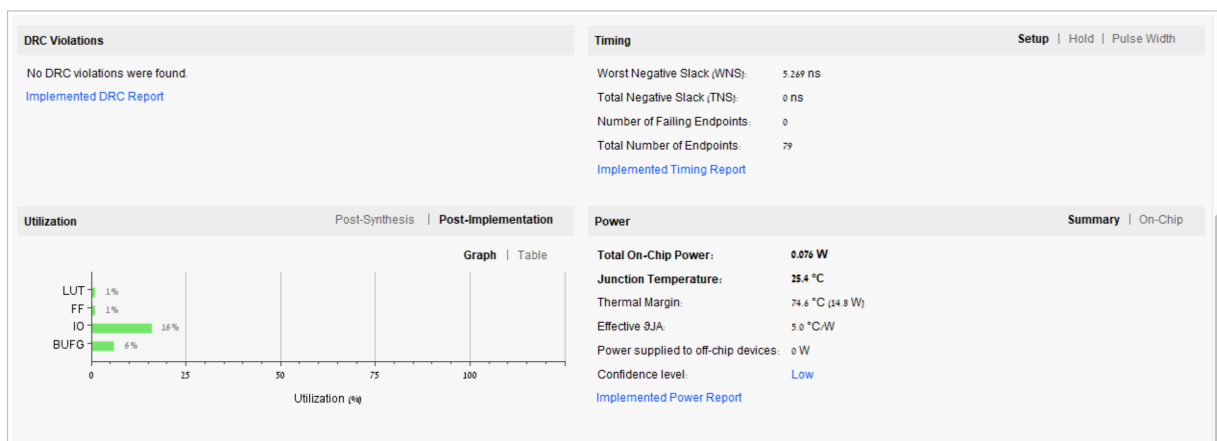
Implementation

Synthesis schematic



Post-synthesis schematic. Every external port now passes through an IBUF/OBUF, the two clocks

Project summary



Post-implementation project summary

Timing summary

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.269 ns		Worst Hold Slack (WHS): 0.330 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 79		Total Number of Endpoints: 79		Total Number of Endpoints: 57	
All user specified timing constraints are met.					

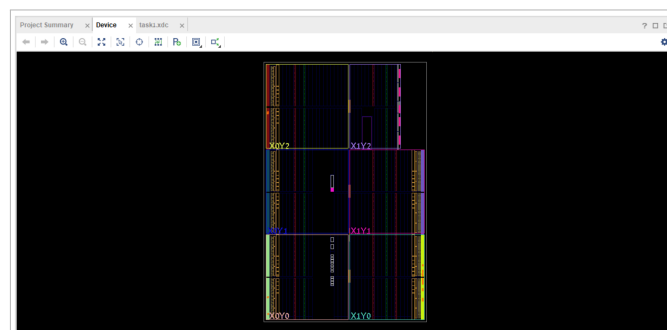
Design Timing Summary after implementation

Critical Path

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock t
Path 1	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[20]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 2	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[21]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 3	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[22]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 4	5.269	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[23]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 5	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[4]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 6	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[5]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 7	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[6]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 8	5.534	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[7]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 9	5.576	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[16]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		
Path 10	5.576	3	25	u_displayled_timer_reg[16]/C	u_displayled_timer_reg[17]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		

Intra-Clock Paths report. The worst paths (Path 1–4) run inside `u_display` from `led_timer_reg[16]/C` to `led_timer_reg[20..23]/CE`, with slack 5.269 ns, 3 logic levels and 4.361 ns total delay (1.054 ns logic + 3.307 ns net) against the 10 ns requirement

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (XC7A35T)

Problems we faced

[paste screenshot here]

Board photo: keypad plugged into the USB host port, a key pressed and its hex code shown on the two right digits with the LED lit.

Appendix — Verilog Source Code

The full source of the submitted files is listed below. Paste the code of each file inside its block.

Ps2_Interface.v

```
1 // --- paste Ps2_Interface.v here ---
```

Ps2_Interface_tb.v

```
1 // --- paste Ps2_Interface_tb.v here ---
```


Ps2_Display.v

```

1  module Ps2_Display(
2      input wire      clk,          // 100 MHz system clock
3      input wire      rstn,         // active-low reset
4      input wire      keyPressed,   // pulse from Ps2_Interface (slow domain)
5      input wire [7:0] scancode,    // byte from Ps2_Interface (slow domain)
6      output wire [6:0] seg,        // 7-segment cathodes (active low)
7      output wire [3:0] an,         // 4 digit anodes (active low)
8      output wire      dp,          // decimal point
9      output reg       led          // strobe LED, one visible blink per
        press
10 );
11 reg [6:0] a_to_g;
12 reg [3:0] an_reg;
13
14 assign seg = a_to_g;
15 assign an  = an_reg;
16 assign dp  = 1'b1;
17
18 wire [1:0] s;
19 reg  [3:0] digit;
20
21 reg kp_sync1, kp_sync2, kp_prev;
22 reg [7:0] valid_scancode;
23
24 always @(posedge clk or negedge rstn) begin
25     if (!rstn) begin
26         kp_sync1 <= 1'b0;
27         kp_sync2 <= 1'b0;
28         kp_prev  <= 1'b0;
29         valid_scancode <= 8'h00;
30     end else begin
31         kp_sync1 <= keyPressed;
32         kp_sync2 <= kp_sync1;
33         kp_prev  <= kp_sync2;
34
35         if (kp_sync2 & ~kp_prev) begin
36             valid_scancode <= scancode;
37         end
38     end
39 end
40
41 wire kp_edge = kp_sync2 & ~kp_prev;
42
43 reg [23:0] led_timer;
44
45 always @(posedge clk or negedge rstn) begin
46     if (!rstn) begin
47         led_timer <= 24'd0;
48         led <= 1'b0;
49     end else if (kp_edge) begin
50         led_timer <= 24'd10_000_000; // ~0.1 seconds at 100MHz
51         led <= 1'b1;
52     end else if (led_timer > 0) begin
53         led_timer <= led_timer - 1;
54         led <= 1'b1;
55     end else begin
56         led <= 1'b0;
57     end
58 end
59
60 reg [19:0] clkdiv = 20'b0;

```

```

61
62     always @(posedge clk or negedge rstn) begin
63         if (!rstn)
64             clkdiv <= 0;
65         else
66             clkdiv <= clkdiv + 1;
67     end
68
69     assign s = clkdiv[19:18];
70
71     always @(*) begin
72         an_reg = 4'b1111;
73
74         case(s)
75             2'b00: begin
76                 digit = valid_scancode[3:0];
77                 an_reg[0] = 1'b0;
78             end
79             2'b01: begin
80                 digit = valid_scancode[7:4];
81                 an_reg[1] = 1'b0;
82             end
83             default: begin
84                 digit = 4'h0;
85                 an_reg = 4'b1111;
86             end
87         endcase
88     end
89
90     always @(*) begin
91         case(digit)
92             //////////<---MSB-LSB<---////////
93             4'h0: a_to_g = 7'b1000000; // 0
94             4'h1: a_to_g = 7'b1111001; // 1
95             4'h2: a_to_g = 7'b0100100; // 2
96             4'h3: a_to_g = 7'b0110000; // 3
97             4'h4: a_to_g = 7'b0011001; // 4
98             4'h5: a_to_g = 7'b0010010; // 5
99             4'h6: a_to_g = 7'b0000010; // 6
100            4'h7: a_to_g = 7'b1111000; // 7
101            4'h8: a_to_g = 7'b0000000; // 8
102            4'h9: a_to_g = 7'b0010000; // 9
103            4'hA: a_to_g = 7'b0001000; // A
104            4'hB: a_to_g = 7'b0000011; // b
105            4'hC: a_to_g = 7'b1000110; // C
106            4'hD: a_to_g = 7'b0100001; // d
107            4'hE: a_to_g = 7'b0000110; // E
108            4'hF: a_to_g = 7'b0001110; // F
109            default: a_to_g = 7'b1111111; // blank
110        endcase
111    end
112 endmodule

```

Ps2_Top.v

```
1 module Ps2_Top(  
2     input wire      clk,           // W5 - 100 MHz system clock  
3     input wire      reset,        // U18 - btnC, active high (pressed = 1)  
4     input wire      PS2Clk,       // C17 - keyboard clock  
5     input wire      PS2Data,      // B17 - keyboard data  
6     output wire [6:0] seg,        // 7-segment cathodes  
7     output wire [3:0] an,         // 7-segment anodes  
8     output wire      dp,          // decimal point  
9     output wire      led          // U16 - strobe LED (LD0)  
10 );  
11  
12 wire rstn = ~reset;  
13 wire keyPressed;  
14 wire [7:0] scancode;  
15  
16 Ps2_Display u_display(  
17     .clk (clk),  
18     .rstn (rstn),  
19     .keyPressed (keyPressed),  
20     .scancode (scancode),  
21     .seg (seg),  
22     .an (an),  
23     .dp (dp),  
24     .led (led));  
25  
26 Ps2_Interface u_interface(  
27     .PS2Clk (PS2Clk),  
28     .rstn (rstn),  
29     .PS2Data (PS2Data),  
30     .scancode (scancode),  
31     .keyPressed (keyPressed));  
32  
33 endmodule
```

task1.xdc

```

1  # Clock signal
2  set_property PACKAGE_PIN W5 [get_ports clk]
3  set_property IOSTANDARD LVCMOS33 [get_ports clk]
4  create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [
    get_ports clk]
5  # LEDs
6  set_property PACKAGE_PIN U16 [get_ports led]
7  set_property IOSTANDARD LVCMOS33 [get_ports led]
8  #7 segment display
9  set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
11 set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
13 set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
15 set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
17 set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
19 set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
21 set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
23
24 set_property PACKAGE_PIN V7 [get_ports dp]
25 set_property IOSTANDARD LVCMOS33 [get_ports dp]
26
27 set_property PACKAGE_PIN U2 [get_ports {an[0]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
29 set_property PACKAGE_PIN U4 [get_ports {an[1]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
31 set_property PACKAGE_PIN V4 [get_ports {an[2]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
33 set_property PACKAGE_PIN W4 [get_ports {an[3]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
35
36
37 #Buttons
38 set_property PACKAGE_PIN U18 [get_ports reset]
39 set_property IOSTANDARD LVCMOS33 [get_ports reset]
40 #USB HID (PS/2)
41 set_property PACKAGE_PIN C17 [get_ports PS2Clk]
42 set_property IOSTANDARD LVCMOS33 [get_ports PS2Clk]
43 set_property PULLUP true [get_ports PS2Clk]
44 set_property PACKAGE_PIN B17 [get_ports PS2Data]
45 set_property IOSTANDARD LVCMOS33 [get_ports PS2Data]
46 set_property PULLUP true [get_ports PS2Data]
47
48 # Don't even think to change the following values !!!!!
49 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
50 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
51 set_property CONFIG_MODE SPIx4 [current_design]
52 set_property CFGBVS VCC0 [current_design]
53 set_property CONFIG_VOLTAGE 3.3 [current_design]

```